How Blink works

bit.ly/how-blink-works
Author: haraken@
Last update: 2018 Aug 14
Status: PUBLIC

Working on Blink is not easy. It's not easy for new Blink developers because there are a lot of Blink-specific concepts and coding conventions that have been introduced to implement a very fast rendering engine. It's not easy even for experienced Blink developers because Blink is huge and extremely sensitive to performance, memory and security.

This document aims at providing a **10k foot overview of "how Blink works"**, which I hope will help Blink developers get familiar with the architecture quickly:

- The document is NOT a thorough tutorial of Blink's detailed architectures and coding rules (which are likely to change and be outdated). Rather the document concisely describes Blink's fundamentals that are not likely to change in the short term and points out resources you can read if you want to learn more.
- The document does NOT explain specific features (e.g., ServiceWorkers, editing). Rather the document explains fundamental features used by a broad range of the code base (e.g., memory management, V8 APIs).

For more general information about Blink's development, see the Chromium wiki page.

What Blink does

Process / thread architecture
- Processes
- Threads
- Initialization of Blink

Directory structure
- Content public APIs and Blink public APIs
- Directory structure and dependencies
- WTF

Memory management

Task scheduling

Page, Frame, Document, DOMWindow etc
- Concepts

Blink 的工作原理

bit.ly/blink 工作原理
作者：haraken@
最后更新时间：2018 年 8 月 14 日
状态：公开

Blink 的开发并不容易。对于新的 Blink 开发者来说，并不容易，因为引入了大量 Blink 特有的概念和编码规范，以实现一个极快的渲染引擎。即使是经验丰富的 Blink 开发者，也不容易，因为 Blink 体量巨大，且对性能、内存和安全性极其敏感。

本文旨在提供关于"Blink 如何工作"的 10k 英尺概览，希望这能帮助 Blink 开发者快速熟悉其架构：

- 本文档并非 Blink 详细架构和编码规则的全面教程（这些内容可能会更改并过时）。本文档简明扼要地描述了 Blink 短期内不太可能改变的基本原则，并指出了您可以阅读以了解更多内容的资源。
- 本文档未解释具体功能（例如 ServiceWorkers、编辑功能）。相反，本文档解释了代码库中广泛使用的核心功能（例如内存管理、V8 API）。

有关 Blink 开发的更多通用信息，请参阅 Chromium 维基页面。

Blink 的功能

进程/线程架构
- 进程
- 线程
- Blink 初始化

目录结构
- 内容公共 API 和 Blink 公共 API
- 目录结构和依赖项
- WTF

内存管理

任务调度

页面、框架、文档、DOM 窗口等
- 概念

- OOPIF
- Detached Frame / Document

Web IDL bindings

V8 and Blink
- Isolate, Context, World
- V8 APIs
- V8 wrappers

Rendering pipeline

Questions?

What Blink does

Blink is a rendering engine of the web platform. Roughly speaking, Blink implements everything that renders content inside a browser tab:

- Implement the specs of the web platform (e.g., HTML standard), including DOM, CSS and Web IDL
- Embed V8 and run JavaScript
- Request resources from the underlying network stack
- Build DOM trees
- Calculate style and layout
- Embed Chrome Compositor and draw graphics

Blink is embedded by many customers such as Chromium, Android WebView and Opera via content public APIs.

- OOPIF
- 独立框架/文档

Web IDL 绑定

V8 和 Blink
- 隔离，上下文，世界
- V8 API
- V8 包装器

渲染管线

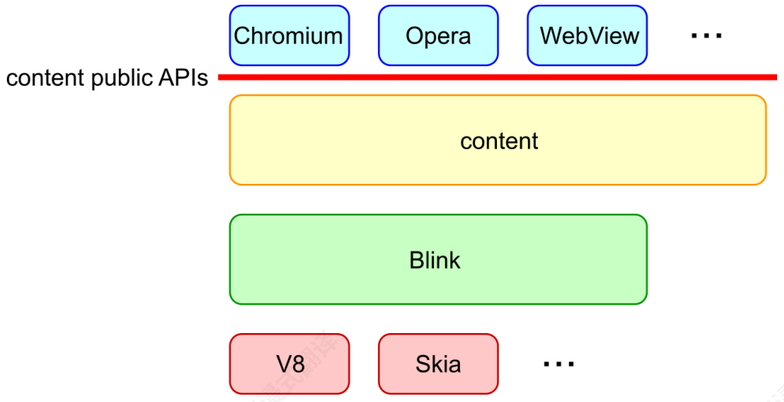
有问题吗?

Blink 的功能

Blink 是 Web 平台的渲染引擎。粗略来说，Blink 实现了浏览器标签页内所有内容渲染所需的一切：

- 实现 Web 平台规范（如 HTML 标准），包括 DOM、CSS 和 Web IDL
- 嵌入 V8 并运行 JavaScript
- 从底层网络栈请求资源
- 构建 DOM 树
- 计算样式和布局
- 嵌入 Chrome 合成器并绘制图形

Blink 被许多客户嵌入，包括通过内容公开 API 的 Chromium、Android WebView 和 Opera。



From the code base perspective, "Blink" normally means //third_party/blink/. From the project perspective, "Blink" normally means projects that implement web platform features. Code that implements web platform features span //third_party/blink/, //content/renderer/, //content/browser/ and other places.

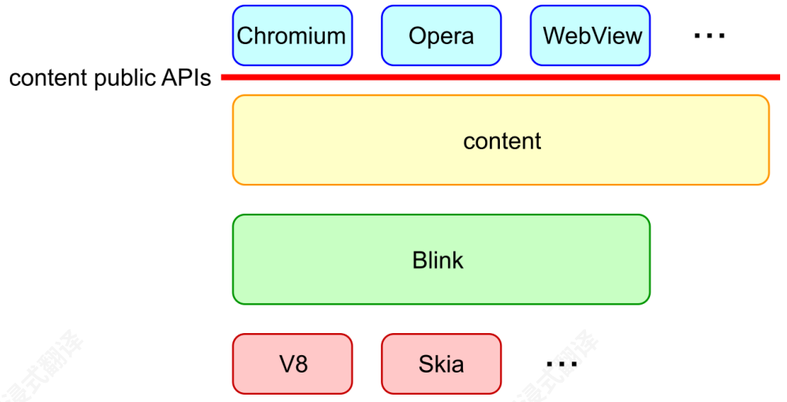
Process / thread architecture

Processes

Chromium has a multi-process architecture. Chromium has one browser process and N sandboxed renderer processes. Blink runs in a renderer process.

How many renderer processes are created? For security reasons, it is important to isolate memory address regions between cross-site documents (this is called Site Isolation). Conceptually each renderer process should be dedicated to at most one site. Realistically, however, it's sometimes too heavy to limit each renderer process to a single site when users open too many tabs or the device does not have enough RAM. Then a renderer process may be shared by multiple iframes or tabs loaded from different sites. This means that iframes in one tab may be hosted by different renderer processes and that iframes in different tabs may be hosted by the same renderer process. **There is no 1:1 mapping between renderer processes, iframes and tabs**.

Given that a renderer process runs in a sandbox, Blink needs to ask the browser process to dispatch system calls (e.g., file access, play audio) and access user profile data (e.g., cookie, passwords). This browser-renderer process communication is realized by Mojo. (Note: In the past we were using Chromium IPC and a bunch of places are still using it. However, it's deprecated and uses Mojo under the hood.) On the Chromium side, Servicification is ongoing



从代码库角度来看，"Blink" 通常指 //third_party/blink/。从项目角度来看，"Blink" 通常指实现 Web 平台功能的项目。实现 Web 平台功能的代码分布在 //third_party/blink/、//content/renderer/、//content/browser/ 等位置。

进程/线程架构

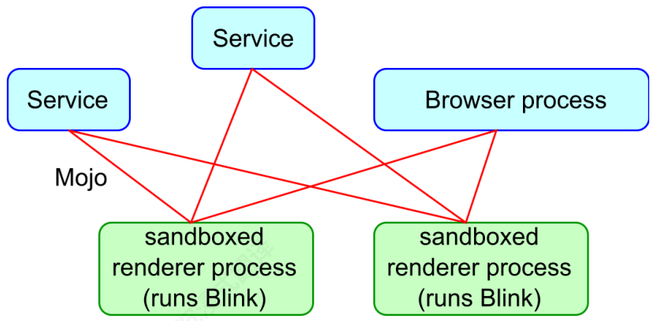
进程

Chromium 采用多进程架构。Chromium 包含一个浏览器进程和 N 个沙盒渲染进程。Blink 在渲染进程中运行。

创建了哪些渲染进程？出于安全原因，需要在跨站点文档之间隔离内存地址区域（这被称为站点隔离）。从概念上讲，每个渲染进程应专用于最多一个站点。然而，现实情况是，当用户打开过多标签页或设备内存不足时，将每个渲染进程限制为仅服务于单个站点可能过于沉重。此时，一个渲染进程可能被多个来自不同站点的 iframe 或标签页共享。这意味着一个标签页中的 iframe 可能由不同的渲染进程托管，而不同标签页中的 iframe 可能由同一个渲染进程托管。渲染进程、iframe 和标签页之间不存在 1:1 的映射关系。

由于渲染进程运行在沙箱中，Blink 需要请求浏览器进程分发系统调用（例如文件访问、播放音频）并访问用户配置文件数据（例如 Cookie、密码）。这种浏览器与渲染进程之间的通信由 Mojo 实现。（注：过去我们曾使用 Chromium IPC，但仍有不少地方仍~~在沿~~用。不过它已被弃用，底层实际~~使用的是 Mojo。~~）在 Chromium 一侧，服务化工作正在进行中。

and abstracting the browser process as a set of "service"s. From the Blink perspective, Blink can just use Mojo to interact with the services and the browser process.



If you want to learn more:

- Multi-process Architecture
- Mojo programming in Blink: platform/mojo/MojoProgrammingInBlink.md

Threads

How many threads are created in a renderer process?

Blink has one main thread, N worker threads and a couple of internal threads.

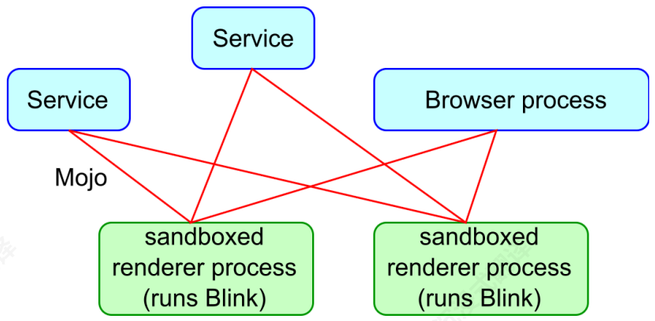
Almost all important things happen on the main thread. All JavaScript (except workers), DOM, CSS, style and layout calculations run on the main thread. Blink is highly optimized to maximize the performance of the main thread, assuming the mostly single-threaded architecture.

Blink may create multiple worker threads to run Web Workers, ServiceWorker and Worklets.

Blink and V8 may create a couple of internal threads to handle webaudio, database, GC etc.

For cross-thread communications, you have to use message passing using PostTask APIs. Shared memory programming is discouraged except for a couple of places that really need to use it for performance reasons. This is why you don't see many MutexLocks in the Blink code base.

并将浏览器进程抽象为一组"服务"。从 Blink 的角度来看，Blink 只需通过 Mojo 与服务及浏览器进程进行交互即可。



如果您想了解更多：

- 多进程架构
- Blink 中的 Mojo 编程：platform/mojo/MojoProgrammingInBlink.md

线程

渲染器进程中会创建多少个线程？

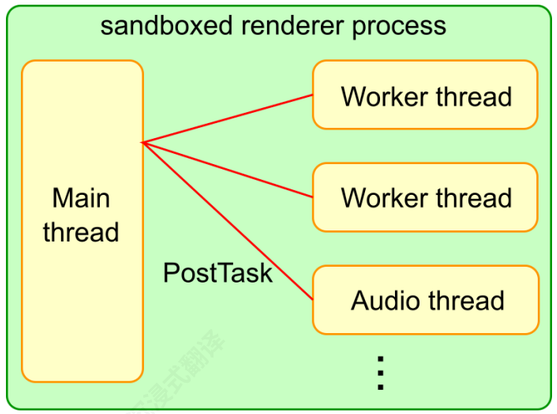
Blink 有一个主线程、N 个工作线程和几个内部线程。

几乎所有重要操作都在主线程上执行。除工作线程外的所有 JavaScript、DOM、CSS、样式和布局计算都在主线程上运行。Blink 针对主线程进行了高度优化，以最大化其性能，这基于其主要是单线程的架构假设。

Blink 可能会创建多个工作线程来运行 Web Workers、ServiceWorker 和 Worklets。

Blink 和 V8 可能会创建几个内部线程来处理 WebAudio、数据库、垃圾回收等。

对于跨线程通信，您必须使用 PostTask API 进行消息传递。除了少数真正需要它以获得性能提升的场景外，不建议使用共享内存编程。这就是为什么在 Blink 代码库中很少看到 MutexLock 的原因。



If you want to learn more:

- Thread programming in Blink: platform/wtf/ThreadProgrammingInBlink.md
- Workers: core/workers/README.md

Initialization & finalization of Blink

Blink is initialized by BlinkInitializer::Initialize(). This method must be called before executing any Blink code.

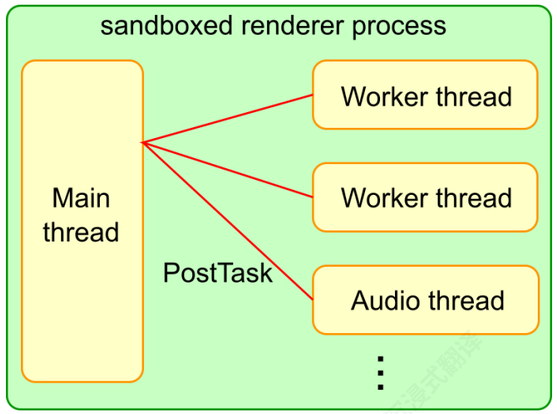
On the other hand, Blink is never finalized; i.e., the renderer process is forcibly exited without being cleaned up. One reason is performance. The other reason is in general it's really hard to clean up everything in the renderer process in a gracefully ordered manner (and it's not worth the effort).

Directory structure

Content public APIs and Blink public APIs

Content public APIs are the API layer that enables embedders to embed the rendering engine. Content public APIs must be carefully maintained because they are exposed to embedders.

Blink public APIs are the API layer that exposes functionalities from //third_party/blink/ to Chromium. This API layer is just a historical artifact inherited from WebKit. In the WebKit era, Chromium and Safari shared the implementation of WebKit, so the API layer was needed to expose functionalities from WebKit to Chromium and Safari. Now that Chromium is the only embedder of //third_party/blink/, the API layer does not make sense. We're actively decreasing



如果您想了解更多：

- Blink 中的线程编程：platform/wtf/ThreadProgrammingInBlink.md
- 工人：core/workers/README.md

Blink 的初始化与最终化

Blink 由 BlinkInitializer::Initialize() 初始化。此方法必须在执行任何 Blink 代码之前调用。

另一方面，Blink 从未真正完成；即渲染进程被强制退出，而未进行清理。原因之一是性能。另一个原因是，通常很难以一种优雅有序的方式清理渲染进程中的所有资源（而且这并不值得付出努力）。

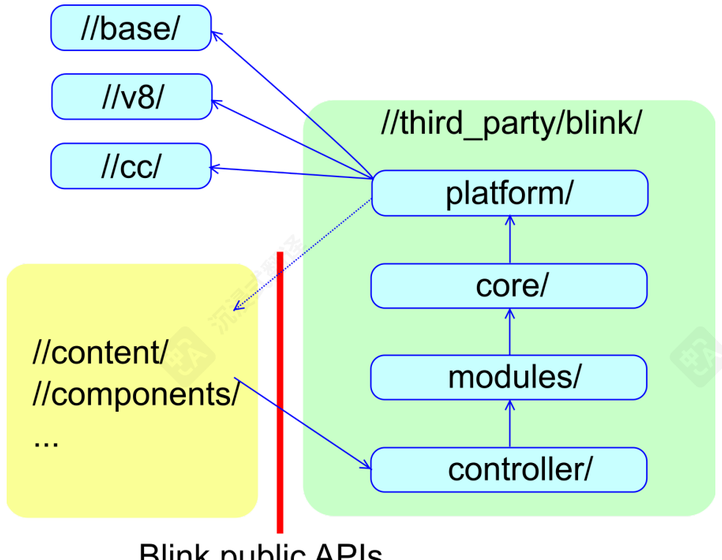
目录结构

内容公共 API 和 Blink 公共 API

内容公开 API 是使嵌入者能够嵌入渲染引擎的 API 层。由于内容公开 API 会暴露给嵌入者，因此必须谨慎维护。

Blink 公共 API 是向 Chromium 暴露 //third_party/blink/ 中功能功能的 API 层。该 API 层仅是从 WebKit 继承下来的历史遗留物。在 WebKit 时代，Chromium 和 Safari 共享 WebKit 的实现，因此需要该 API 层将 WebKit 的功能暴露给 Chromium 和 Safari。现在 Chromium 是 //third_party/blink/ 的唯一嵌入方，该 API 层已无意义。我们正积极减少

of Blink public APIs by moving web-platform code from Chromium to Blink (the project is called Onion Soup).



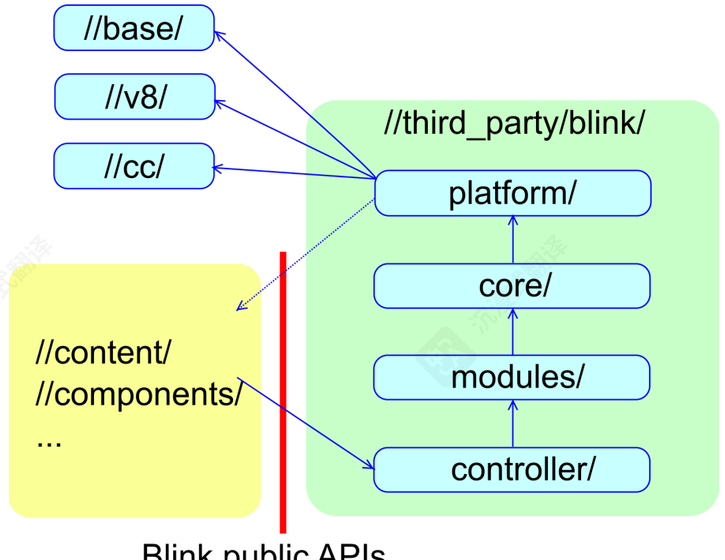
Directory structure and dependencies

//third_party/blink/ has the following directories. See this document for a more detailed definition of these directories:

- platform/
  - A collection of lower level features of Blink that are factored out of a monolithic core/. e.g., geometry and graphics utils.
- core/ and modules/
  - The implementation of all web-platform features defined in the specs. core/ implements features tightly coupled with DOM. modules/ implements more self-contained features. e.g. webaudio, indexeddb.
- bindings/core/ and bindings/modules/
  - Conceptually bindings/core/ is part of core/, and bindings/modules/ is part of modules/. Files that heavily use V8 APIs are put in bindings/{core,modules}.
- controller/
  - A set of high-level libraries that use core/ and modules/. e.g., devtools front-end.

Dependencies flow in the following order:

通过从 Chromium 迁移到 Blink（该项目名为 Onion Soup）来增加 Blink 的公开 API 数量。



目录结构和依赖项

//third_party/blink/ 包含以下目录。有关这些目录的更详细定义，请参阅本文档：

- 平台/
  - Blink 中从单一核心/中分离出来的底层功能集合。例如，几何和图形工具。
- core/ 和 modules/
  - 实现所有在规范中定义的 Web 平台功能。core/ 实现与 DOM 紧密耦合的功能。modules/ 实现更自包含的功能，例如 WebAudio、IndexedDB。
- bindings/core/ 和 bindings/modules/
  - 概念上，bindings/core/ 是 core/ 的一部分，bindings/modules/ 是 modules/ 的一部分。大量使用 V8 API 的文件应放在 bindings/{core,modules} 中。
- controller/
  - 一套使用 core/ 和 modules/ 的高级库。例如：devtools 前端。

依赖项按以下顺序流动：

- Chromium => controller/ => modules/ and bindings/modules/ => core/ and bindings/core/ => platform/ => low-level primitives such as //base, //v8 and //cc

Blink carefully maintains the list of low-level primitives exposed to //third_party/blink/.

If you want to learn more:

- Directory structure and dependencies: blink/renderer/README.md

WTF

WTF is a "Blink-specific base" library and located at platform/wtf/. We are trying to unify coding primitives between Chromium and Blink as much as possible, so WTF should be small. This library is needed because there are a number of types, containers and macros that really need to be optimized for Blink's workload and Oilpan (Blink GC). If types are defined in WTF, Blink has to use the WTF types instead of types defined in //base or std libraries. The most popular ones are vectors, hashsets, hashmaps and strings. Blink should use WTF::Vector, WTF::HashSet, WTF::HashMap, WTF::String and WTF::AtomicString instead of std::vector, std::*set, std::*map and std::string.

If you want to learn more:

- How to use WTF: platform/wtf/README.md

Memory management

As far as Blink is concerned, you need to care about three memory allocators:

- PartitionAlloc
- Oil Pan (a.k.a. Blink GC)
- malloc/free or new/delete (banned)

You can allocate an object on PartitionAlloc's heap by using USING_FAST_MALLOC():

```
class SomeObject {
  USING_FAST_MALLOC(SomeObject);
  static std::unique_ptr<SomeObject> Create() {
    return std::make_unique<SomeObject>();  // Allocated on
PartitionAlloc's heap.
  }
};
```

● Chromium => controller/ => modules/ 和 bindings/modules/ => core/ 和 bindings/core/=> platform/ => 低层原语，如 //base、//v8 和 //cc

Blink 谨慎地维护着暴露给 //third_party/blink/ 的低级原语列表。

如果您想了解更多：

● 目录结构和依赖：blink/renderer/README.md

WTF

WTF 是一个"Blink 专用基础”库，位于 platform/wtf/ 目录下。我们正努力尽可能统一 Chromium 和 Blink 之间的编码原语，因此 WTF 应保持精简。需要此库是因为存在大量需要针对 Blink 的工作负载和 Oilpan（Blink 垃圾回收器）进行优化的类型、容器和宏。如果类型在 WTF 中定义，Blink 就必须使用 WTF 的类型，而不是 //base 或标准库中定义的类型。其中最常用的包括向量、哈希集合、哈希映射和字符串。Blink 应使用 WTF::Vector、WTF::HashSet、WTF::HashMap、WTF::String 和 WTF::AtomicString，而不是 std::vector、std::*set、std::*map 和 std::string。

如果您想了解更多：

● 如何使用 WTF：platform/wtf/README.md

内存管理

就 Blink 而言，您需要关注三个内存分配器：

- 分区分配器
- 曲轴箱（又名 Blink GC）
- malloc/free 或 new/delete（禁止）

您可以使用 USING_FAST_MALLOC() 在 PartitionAlloc 的堆上分配对象：

```
class SomeObject {
  USING_FAST_MALLOC(SomeObject);
  static std::unique_ptr<SomeObject> Create() {
    return std::make_unique<SomeObject>();  // Allocated on
PartitionAlloc's heap.
  }
};
```

The lifetime of objects allocated by PartitionAlloc should be managed by scoped_refptr<> or std::unique_ptr<>. It is strongly discouraged to manage the lifetime manually. Manual `delete` is banned in Blink.

You can allocate an object on Oilpan's heap by using GarbageCollected:

```
class SomeObject : public GarbageCollected<SomeObject> {
  static SomeObject* Create() {
    return new SomeObject;  // Allocated on Oilpan's heap.
  }
};
```

The lifetime of objects allocated by Oilpan is automatically managed by garbage collection. You have to use special pointers (e.g., Member<>, Persistent<>) to hold objects on Oilpan's heap. See this API reference to get familiar with programming restrictions about Oilpan. The most important restriction is that you are not allowed to touch any other Oilpan's object in a destructor of Oilpan's object (because the destruction order is not guaranteed).

If you use neither USING_FAST_MALLOC() nor GarbageCollected, objects are allocated on system malloc's heap. This is strongly discouraged in Blink. All Blink objects should be allocated by PartitionAlloc or Oilpan, as follows:

- Use Oilpan by default.
- Use PartitionAlloc only when 1) the lifetime of the object is very clear and std::unique_ptr<> or scoped_refptr<> is enough, 2) allocating the object on Oilpan introduces a lot of complexity or 3) allocating the object on Oilpan introduces a lot of unnecessary pressure to the garbage collection runtime.

Regardless of whether you use PartitionAlloc or Oilpan, you have to be really careful not to create dangling pointers (Note: raw pointers are strongly discouraged) or memory leaks.

If you want to learn more:

- How to use PartitionAlloc: platform/wtf/allocator/Allocator.md
- How to use Oilpan: platform/heap/BlinkGCAPIReference.md
- Oilpan GC design: platform/heap/BlinkGCDesign.md

Task scheduling

To improve responsiveness of the rendering engine, tasks in Blink should be executed asynchronously whenever possible. Synchronous IPC / Mojo and any other operations that may

由 PartitionAlloc 分配的对象的生命周期应由 scoped_refptr<> 或 std::unique_ptr<> 管理。强烈不建议手动管理生命周期。Blink 禁止手动 delete。

您可以使用 GarbageCollected 在 Oilpan 的堆上分配对象：

class SomeObject : public GarbageCollected<SomeObject> { static SomeObject* Create() { return new SomeObject; // 在 Oilpan 的堆上分配。} };

Oilpan 分配的对象的寿命由垃圾回收自动管理。您必须使用特殊指针（例如 Member<>、Persistent<>）来持有 Oilpan 堆上的对象。请参阅此 API 参考，以了解 Oilpan 的编程限制。最重要的限制是：您不允许在 Oilpan 对象的析构函数中访问任何其他 Oilpan 对象（因为析构顺序无法保证）。

如果您既不使用 USING_FAST_MALLOC() 也不使用 GarbageCollected，则对象将在系统 malloc 的堆上分配。在 Blink 中强烈不推荐这样做。所有 Blink 对象都应由 PartitionAlloc 或 Oilpan 分配，如下所示：

- 默认使用 Oilpan。
- 仅在满足以下条件时使用 PartitionAlloc：1) 对象的生命周期非常明确，std::unique_ptr<> 或 scoped_refptr<> 已足够；2) 在 Oilpan 上分配该对象会引入大量复杂性；或 3) 在 Oilpan 上分配该对象会给垃圾回收运行时带来大量不必要的压力。

无论您使用 PartitionAlloc 还是 Oilpan，都必须非常小心，避免创建悬空指针（注意：原始指针强烈不推荐使用）或内存泄漏。

如果您想了解更多：

- 如何使用 PartitionAlloc：platform/wtf/allocator/Allocator.md
- 如何使用 Oilpan：platform/heap/BlinkGCAPIReference.md
- Oilpan GC 设计：平台/堆/BlinkGCDesign.md

任务调度

为提升渲染引擎的响应性，Blink 中的任务应尽可能异步执行。同步 IPC/Mojo 以及任何其他可能

take several milliseconds are discouraged (although some are inevitable e.g., user's JavaScript execution).

All tasks in a renderer process should be posted to Blink Scheduler with proper task types, like this:

```
// Post a task to frame's scheduler with a task type of kNetworking
frame->GetTaskRunner(TaskType::kNetworking)->PostTask(...,
WTF::Bind(&Function));
```

Blink Scheduler maintains multiple task queues and smartly prioritizes tasks to maximize user-perceived performance. It is important to specify proper task types to let Blink Scheduler schedule the tasks correctly and smartly.

If you want to learn more:

- How to post tasks: third_party/blink/renderer/platform/scheduler/TaskSchedulingInBlink.md

Page, Frame, Document, DOMWindow etc

Concepts

Page, Frame, Document, ExecutionContext and DOMWindow are the following concepts:

- A Page corresponds to a concept of a tab (if OOPIF explained below is not enabled). Each renderer process may contain multiple tabs.
- A Frame corresponds to a concept of a frame (the main frame or an iframe). Each Page may contain one or more Frames that are arranged in a tree hierarchy.
- A DOMWindow corresponds to a `window` object in JavaScript. Each Frame has one DOMWindow.
- A Document corresponds to a `window.document` object in JavaScript. Each Frame has one Document.
- An ExecutionContext is a concept that abstracts a Document (for the main thread) and a WorkerGlobalScope (for a worker thread).

Renderer process : Page = 1 : N.

Page : Frame = 1 : M.

Frame : DOMWindow : Document (or ExecutionContext) = 1 : 1 : 1 at any point in time, but the mapping may change over time. For example, consider the following code:

不建议花费几毫秒（尽管有些是不可避免的，例如用户的 JavaScript 执行）。

渲染器中的所有任务都应使用正确的任务类型提交到 Blink 调度器，例如：

```
// 向 frame 的调度器发布任务，任务类型为 kNetworking frame->
GetTaskRunner(TaskType::kNetworking)->PostTask(..., WTF::Bind(&Function));
```

Blink Scheduler 维护多个任务队列，并智能地优先处理任务，以最大化用户感知的性能。正确指定任务类型对于让 Blink Scheduler 正确且智能地调度任务至关重要。

如果您想了解更多：

- 如何发布任务：third_party/blink/renderer/platform/scheduler/TaskSchedulingInBlink.md

页面、框架、文档、DOM 窗口等

概念

Page、Frame、Document、ExecutionContext 和 DOMWindow 是以下概念：

- 页面对应于标签页的概念（如果下方未启用 OOPIF）。每个渲染进程可包含多个标签页。

● 一个 Frame 对应于一个帧的概念（主帧或内嵌帧）。每个 Page 可以包含一个或多个以树形层次结构排列的 Frame。
● DOMWindow 对应 JavaScript 中的 window 对象。每个 Frame 包含一个 DOMWindow。
● 一个 Document 对应于 JavaScript 中的 window.document 对象。每个 Frame 都有一个 Document。

- ExecutionContext 是一个抽象概念，用于封装主线程中的 Document 和 worker 线程中的 WorkerGlobalScope。

渲染器进程：页面 = 1 ： N。

页面：帧 = 1 ： M。

Frame : DOMWindow : Document (或 ExecutionContext) = 1 : 1 : 1 在任意时刻，但映射可能会随时间变化。例如，考虑以下代码：

```
iframe.contentWindow.location.href = "https://example.com";
```

In this case, a new DOMWindow and a new Document are created for https://example.com. However, the Frame may be reused.

(Note: Precisely speaking, there are some cases where a new Document is created but the DOMWindow and the Frame are reused. There are even more complex cases.)

If you want to learn more:

- core/frame/FrameLifecycle.md

Out-of-Process iframes (OOPIF)

Site Isolation makes things more secure but even more complex. :) The idea of Site Isolation is to create one renderer process per site. (A *site* is a page's registrable domain + 1 label, and its URL scheme. For example, https://mail.example.com and https://chat.example.com are in the same site, but https://noodles.com and https://pumpkins.com are not.) If a Page contains one cross-site iframe, the Page may be hosted by two renderer processes. Consider the following page:

```
<!-- https://example.com -->
<body>
<iframe src="https://example2.com"></iframe>
</body>
```

The main frame and the <iframe> may be hosted by different renderer processes. A frame local to the renderer process is represented by LocalFrame and a frame not local to the renderer process is represented by RemoteFrame.

From the perspective of the main frame, the main frame is a LocalFrame and the <iframe> is a RemoteFrame. From the perspective of the <iframe>, the main frame is a RemoteFrame and the <iframe> is a LocalFrame.

Communications between a LocalFrame and RemoteFrame (which may exist in different renderer processes) are handled via the browser process.

If you want to learn more:

- Design docs: Site isolation design docs
- How to write code with site isolation: core/frame/SiteIsolation.md

```
iframe.contentWindow.location.href = "https://example.com";
```

在这种情况下，为 https://example.com 创建一个新的 DOMWindow 和一个新的 Document。但是，Frame 可能会被重用。

（注：严格来说，存在一些情况会创建新的 Document，但重用 DOMWindow 和 Frame。甚至存在更复杂的情况。）

如果您想了解更多：

- core/frame/FrameLifecycle.md

独立进程 iframe（OOPIF）

站点隔离虽然能提升安全性，但也让事情变得更加复杂。:) 站点隔离的核心思想是为每个站点创建一个渲染进程。（站点是指一个页面的可注册域名 + 1 label 及其 URL 协议。例如，https://mail.example.com 和 https://chat.example.com 属于同一站点，但 https://noodles.com 和 https://pumpkins.com 则不属于同一站点。）如果某个页面包含一个跨站点的 iframe，该页面可能由两个渲染进程托管。考虑以下页面：

```
<!-- https://example.com --><body><iframe src="
https://example2.com"></iframe></body>
```

主框架和 <iframe> 可能由不同的渲染进程托管。本地于渲染进程的框架由 LocalFrame 表示，而不在渲染进程中本地的框架由 RemoteFrame 表示。

从主框架的视角来看，主框架是一个 LocalFrame，而 <iframe> 是一个 RemoteFrame。从 <iframe> 的视角来看，主框架是一个 RemoteFrame，而 <iframe> 是一个 LocalFrame。

本地帧（LocalFrame）与远程帧（RemoteFrame，可能存在于不同的渲染进程中）之间的通信通过浏览器进程处理。

如果您想了解更多：

- 设计文档：站点隔离设计文档
- 如何编写具有站点隔离的代码：core/frame/SiteIsolation.md

Detached Frame / Document

Frame / Document may be in a detached state. Consider the following case:

```
doc = iframe.contentDocument;
iframe.remove();  // The iframe is detached from the DOM tree.
doc.createElement("div");  // But you still can run scripts on the
detached frame.
```

The tricky fact is that you can still run scripts or DOM operations on the detached frame. Since the frame has already been detached, most DOM operations will fail and throw errors. Unfortunately, behaviors on detached frames are not really interoperable among browsers nor well-defined in the specs. Basically the expectation is that JavaScript should keep running but most DOM operations should fail with some proper exceptions, like this:

```
void someDOMOperation(...) {
  if (!script_state_->ContextIsValid()) { // The frame is already
detached
    ...;  // Set an exception etc
    return;
  }
}
```

This means that in common cases Blink needs to do a bunch of clean-up operations when the frame gets detached. You can do this by inheriting from ContextLifecycleObserver, like this:

```
class SomeObject : public GarbageCollected<SomeObject>, public
ContextLifecycleObserver {
  void ContextDestroyed() override {
    // Do clean-up operations here.
  }
  ~SomeObject() {
    // It's not a good idea to do clean-up operations here because it's
too late to do them. Also a destructor is not allowed to touch any
other objects on Oilpan's heap.
  }
};
```

独立框架/文档

帧/文档可能处于分离状态。考虑以下情况：

```
doc = iframe.contentDocument;
iframe.remove(); // 从 DOM 树中移除 iframe。doc.createElement("div"); // 但
您仍然可以在分离的框架上运行脚本。
```

棘手之处在于，您仍然可以在分离的框架上运行脚本或执行 DOM 操作。由于框架已经分离，大多数 DOM 操作将失败并抛出错误。不幸的是，分离框架上的行为在不同浏览器之间并不真正兼容，且在规范中定义也不明确。基本上，预期是 JavaScript 应继续运行，但大多数 DOM 操作应以适当的异常失败，例如：

```
void someDOMOperation(...) { if (!script_state_->ContextIsValid()) { // 该帧已
分离 ...; // 设置异常等 return; } }
```

这意味着在常见情况下，当帧被分离时，Blink 需要执行一系列清理操作。您可以通过继承 ContextLifecycleObserver 来实现这一点，如下所示：

```
class SomeObject : public GarbageCollected<SomeObject>, public
ContextLifecycleObserver {
  void ContextDestroyed() override {
    // Do clean-up operations here.
  }
  ~SomeObject() {
    // It's not a good idea to do clean-up operations here because it's
too late to do them. Also a destructor is not allowed to touch any
other objects on Oilpan's heap.
  }
};
```

Web IDL bindings

When JavaScript accesses `node.firstChild`, `Node::firstChild()` in node.h gets called. How does it work? Let's take a look at how `node.firstChild` works.

First of all, you need to define an IDL file per the spec:

```
// node.idl
interface Node : EventTarget {
  [...] readonly attribute Node? firstChild;
};
```

The syntax of Web IDL is defined in the Web IDL spec. `[...]` is called IDL extended attributes. Some of the IDL extended attributes are defined in the Web IDL spec and others are Blink-specific IDL extended attributes. Except the Blink-specific IDL extended attributes, IDL files should be written in a spec-conformant manner (i.e., just copy and paste from the spec).

Second, you need to define a C++ class for Node and implement a C++ getter for firstChild:

```
class EventTarget : public ScriptWrappable {  // All classes exposed to
JavaScript must inherit from ScriptWrappable.
  ...;
};

class Node : public EventTarget {
  DEFINE_WRAPPERTYPEINFO();  // All classes that have IDL files must
have this macro.
  Node* firstChild() const { return first_child_; }
};
```

In common cases, that's it. When you build node.idl, the IDL compiler auto-generates Blink-V8 bindings for the Node interface and Node.firstChild. The auto-generated bindings are generated in //src/out/{Debug,Release}/gen/third_party/ blink/renderer/bindings/core/v8/v8_node.h. When JavaScript calls `node.firstChild`, V8 calls V8Node::firstChildAttributeGetterCallback() in v8_node.h, then it calls Node::firstChild() which you defined in the above.

If you want to learn more:

- How to add Web IDL bindings: bindings/IDLCompiler.md
- How to use IDL extended attributes: bindings/IDLExtendedAttributes.md
- Spec: Web IDL spec

Web IDL 绑定

当 JavaScript 访问 node.firstChild 时，node.h 中的 Node::firstChild() 会被调用。它是如何工作的？让我们看看 node.firstChild 是如何工作的。

首先，您需要按照规范定义一个 IDL 文件：

```
// node.idl interface Node : EventTarget {[...]
readonly attribute Node? firstChild; };
```

Web IDL 的语法定义在 Web IDL 规范中。`[...]` 称为 IDL 扩展属性。部分 IDL 扩展属性定义在 Web IDL 规范中，其他为 Blink 特有的 IDL 扩展属性。除 Blink 特有的 IDL 扩展属性外，IDL 文件应严格按照规范编写（即直接从规范中复制粘贴）。

其次，您需要为 Node 定义一个 C++ 类，并为 firstChild 实现一个 C++ getter：

```
class EventTarget : public ScriptWrappable { // 所有暴露给 JavaScript 的类都必须继
承自 ScriptWrappable。
  ...;
};

class Node : public EventTarget {
    DEFINE_WRAPPERTYPEINFO(); // 所有拥有 IDL 文件的类都必须包含此宏。

  Node* firstChild() const { return first_child_; }
};
```

在常见情况下，仅此而已。当你构建 node.idl 时，IDL 编译器会自动为 Node 接口和 Node.firstChild 生成 Blink-V8 绑定。这些自动生成的绑定位于 //src/out/{Debug,Release}/gen/third_party/blink/renderer/bindings/core/v8/v8_node.h。当 JavaScript 调用 node.firstChild 时，V8 会在 v8_node.h 中调用 V8Node::firstChildAttributeGetterCallback()，然后调用你在此处定义的 Node::firstChild()。

如果您想了解更多：

- 如何添加 Web IDL 绑定：bindings/IDLCompiler.md
- 如何使用 IDL 扩展属性：bindings/IDLExtendedAttributes.md
- 规范：Web IDL 规范

V8 and Blink

Isolate, Context, World

When you write code that touches V8 APIs, it is important to understand the concept of Isolate, Context and World. They are represented by v8::Isolate, v8::Context and DOMWrapperWorld in the code base respectively.

Isolate corresponds to a physical thread. Isolate : physical thread in Blink = 1 : 1. The main thread has its own Isolate. A worker thread has its own Isolate.

Context corresponds to a global object (In case of a Frame, it's a `window` object of the Frame). Since each frame has its own `window` object, there are multiple Contexts in a renderer process. When you call V8 APIs, you have to make sure that you're in the correct context. Otherwise, v8::Isolate::GetCurrentContext() will return a wrong context and in the worst case it will end up leaking objects and causing security issues.

World is a concept to support content scripts of Chrome extensions. Worlds do not correspond to anything in web standards. Content scripts want to share DOM with the web page, but for security reasons JavaScript objects of content scripts must be isolated from the JavaScript heap of the web page. (Also a JavaScript heap of one content script must be isolated from a JavaScript heap of another content script.) To realize the isolation, the main thread creates one main world for the web page and one isolated world for each content script. The main world and the isolated worlds can access the same C++ DOM objects but their JavaScript objects are isolated. This isolation is realized by creating multiple V8 wrappers for one C++ DOM object; i.e., one V8 wrapper per world.

V8 和 Blink

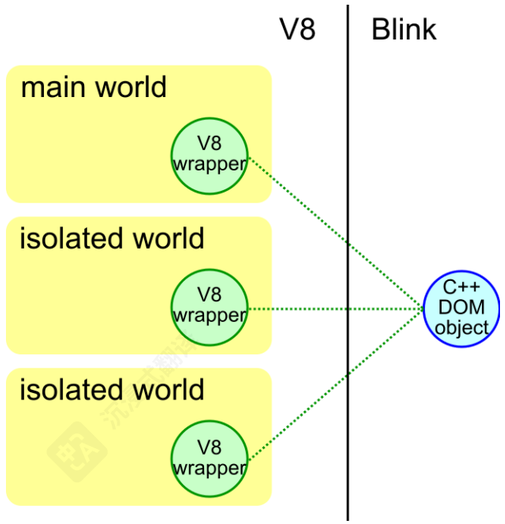
隔离，上下文，世界

当您编写涉及 V8 API 的代码时，理解 Isolate、Context 和 World 的概念非常重要。在代码库中，它们分别由 v8::Isolate、v8::Context 和 DOMWrapperWorld 表示。

Isolate 对应于一个物理线程。Isolate : Blink 中的物理线程 = 1 : 1. 主线程拥有自己的 Isolate。工作线程拥有自己的 Isolate。

Context 对应于一个全局对象（如果是 Frame，则对应 Frame 的 window 对象）。由于每个 frame 都有自己的 window 对象，因此 renderer 进程中存在多个 Context。当您调用 V8 API 时，必须确保您处于正确的上下文中。否则，v8::Isolate::GetCurrentContext() 将返回错误的上下文，在最坏的情况下会导致对象泄漏并引发安全问题。

World 是用于支持 Chrome 扩展内容脚本的概念。Worlds 并不对应任何 Web 标准中的内容。内容脚本希望与网页共享 DOM，但出于安全原因，内容脚本的 JavaScript 对象必须与网页的 JavaScript 堆隔离。（同样，一个内容脚本的 JavaScript 堆也必须与另一个内容脚本的 JavaScript 堆隔离。）为了实现这种隔离，主线程会为网页创建一个主 World，并为每个内容脚本创建一个隔离 World。主 World 和隔离 World 可以访问相同的 C++ DOM 对象，但它们的 JavaScript 对象是隔离的。这种隔离是通过为同一个 C++ DOM 对象创建多个 V8 包装器来实现的；即每个 World 对应一个 V8 包装器。



What's a relationship between Context, World and Frame?

Imagine that there are N Worlds on the main thread (one main world + (N - 1) isolated worlds). Then one Frame should have N `window` objects, each of which is used for one world. Context is a concept that corresponds to a `window` object. This means that when we have M Frames and N Worlds, we have M * N Contexts (but the Contexts are created lazily).

In case of a worker, there is only one World and one global object. Thus there is only one Context.

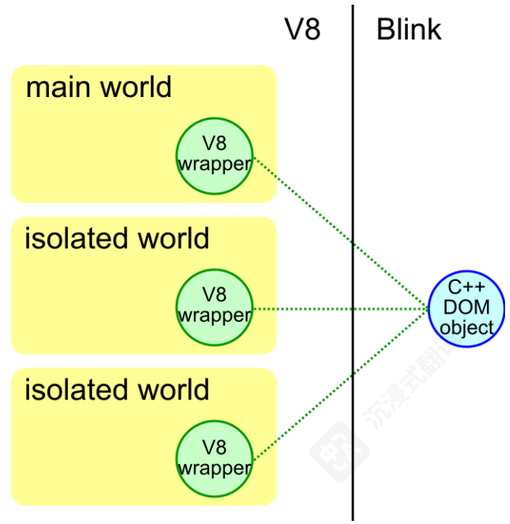
Again, when you use V8 APIs, you should be really careful about using the correct context. Otherwise you'll end up leaking JavaScript objects between isolated worlds and causing security disasters (e.g., an extension from A.com can manipulate an extension from B.com).

If you want to learn more:

- bindings/core/v8/V8BindingDesign.md

V8 APIs

There are a lot of V8 APIs defined in //v8/include/v8.h. Since V8 APIs are low-level and hard to use correctly, platform/bindings/ provides a bunch of helper classes that wrap V8 APIs. You should consider using the helper classes as much as possible. If your code has to use V8 APIs heavily, the files should be put in bindings/{core,modules}.



Context、World 和 Frame 之间的关系是什么？

假设主线程上有 N 个世界（一个主世界 + (N - 1) 个隔离世界）。那么每一帧应包含 N 个窗口对象，每个窗口对象对应一个世界。上下文是与窗口对象对应的概念。这意味着当有 M 帧和 N 个世界时，我们有 M * N 个上下文（但上下文是按需创建的）。

对于 Worker，只有一个 World 和一个全局对象。因此只有一个 Context。

同样，在使用 V8 API 时，务必谨慎使用正确的上下文。否则，您可能会在隔离的世界之间泄露 JavaScript 对象，从而导致安全灾难（例如，来自 A.com 的扩展可以操纵来自 B.com 的扩展）。

如果您想了解更多：

- bindings/core/v8/V8BindingDesign.md

V8 API

在 //v8/include/v8.h 中定义了许多 V8 API。由于 V8 API 属于底层接口且不易正确使用，platform/bindings/ 提供了一系列用于封装 V8 API 的辅助类。您应尽可能使用这些辅助类。如果您的代码需要大量使用 V8 API，则相关文件应放置在 bindings/{core,modules} 目录中。

V8 uses handles to point to V8 objects. The most common handle is v8::Local<>, which is used to point to V8 objects from a machine stack. v8::Local<> must be used after allocating v8::HandleScope on the machine stack. v8::Local<> should not be used outside the machine stack:

```
void function() {
  v8::HandleScope scope;
  v8::Local<v8::Object> object = ...;  // This is correct.
}

class SomeObject : public GarbageCollected<SomeObject> {
  v8::Local<v8::Object> object_;  // This is wrong.
};
```

If you want to point to V8 objects from outside the machine stack, you need to use wrapper tracing. However, you have to be really careful not to create a reference cycle with it. In general V8 APIs are hard to use. Ask blink-review-bindings@ if you're not sure about what you're doing.

If you want to learn more:

- How to use V8 APIs and helper classes: platform/bindings/HowToUseV8FromBlink.md

V8 wrappers

Each C++ DOM object (e.g., Node) has its corresponding V8 wrapper. Precisely speaking, each C++ DOM object has its corresponding V8 wrapper per world.

V8 wrappers have strong references to their corresponding C++ DOM objects. However, the C++ DOM objects have only weak references to the V8 wrappers. So if you want to keep V8 wrappers alive for a certain period of time, you have to do that explicitly. Otherwise, V8 wrappers will be prematurely collected and JS properties on the V8 wrappers will be lost...

```
div = document.getElementbyId("div");
child = div.firstChild;
child.foo = "bar";
child = null;
gc();  // If we don't do anything, the V8 wrapper of |firstChild| is
collected by the GC.
assert(div.firstChild.foo === "bar");  //...and this will fail.
```

If we don't do anything, `child` is collected by the GC and thus `child.foo` is lost. To keep the V8 wrapper of `div.firstChild` alive, we have to add a mechanism that "keeps the V8 wrapper of `div.firstChild` alive as long as the DOM tree which `div` belongs to is reachable from V8".

V8 使用句柄来指向 V8 对象。最常见的句柄是 v8::Local<>，它用于从机器栈指向 V8 对象。v8::Local<> 必须在机器栈上分配 v8::HandleScope 之后使用。v8::Local<> 不应在机器栈之外使用：

```
void function() {
  v8::HandleScope scope;
  v8::Local<v8::Object> object = ...;  // This is correct.
}

class SomeObject : public GarbageCollected<SomeObject> {
  v8::Local<v8::Object> object_;  // This is wrong.
};
```

如果您想从机器栈外部引用 V8 对象，需要使用包装器追踪。但在使用时务必小心，避免创建引用循环。总体而言，V8 API 较难使用。如果您不确定自己的操作，请询问 blink-review-bindings@。

如果您想了解更多：

- 如何使用 V8 API 和辅助类：platform/bindings/HowToUseV8FromBlink.md

V8 包装器

每个 C++ DOM 对象（例如 Node）都有对应的 V8 包装器。更准确地说，每个 C++ DOM 对象都有对应于每个世界的 V8 包装器。

V8 包装器对其对应的 C++ DOM 对象拥有强引用。然而，C++ DOM 对象仅对 V8 包装器拥有弱引用。因此，如果您希望让 V8 包装器在特定时间内保持存活，必须显式执行此操作。否则，V8 包装器将被过早回收，导致 V8 包装器上的 JS 属性丢失……

```
div = document.getElementbyId("div");
child = div.firstChild;
child.foo = "bar";
child = null;
gc(); // 如果不做任何操作，|firstChild| is 将由 GC 回收。

assert(div.firstChild.foo === "bar"); //...并且这将失败。
```

如果我们什么都不做，子节点会被 GC 回收，因此 child.foo 会丢失。为了保持 div.firstChild 的 V8 包装器存活，我们需要添加一种机制，使得"div.firstChild 的 V8 包装器在 div 所属的 DOM 树从 V8 可达时保持存活”。

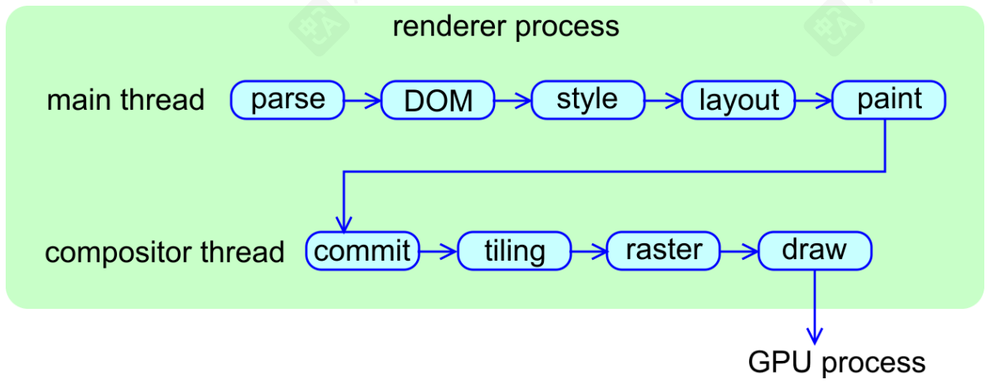
There are two ways to keep V8 wrappers alive: ActiveScriptWrappable and wrapper tracing.

If you want to learn more:

- How to manage lifetime of V8 wrappers: bindings/core/v8/V8Wrapper.md
- How to use wrapper tracing: platform/bindings/TraceWrapperReference.md

Rendering pipeline

There is a long journey from when an HTML file is delivered to Blink to when pixels are displayed on the screen. The rendering pipeline is architectured as follows.



Read this excellent deck to learn what each phase of the rendering pipeline does. (I don't think I can write a better explanation than the deck :-)

If you want to learn more:

- Overview: Life of a pixel
- DOM: core/dom/README.md
- Style: core/css/README.md
- Layout: core/layout/README.md
- Paint: core/paint/README.md
- Compositor thread: Chromium graphics

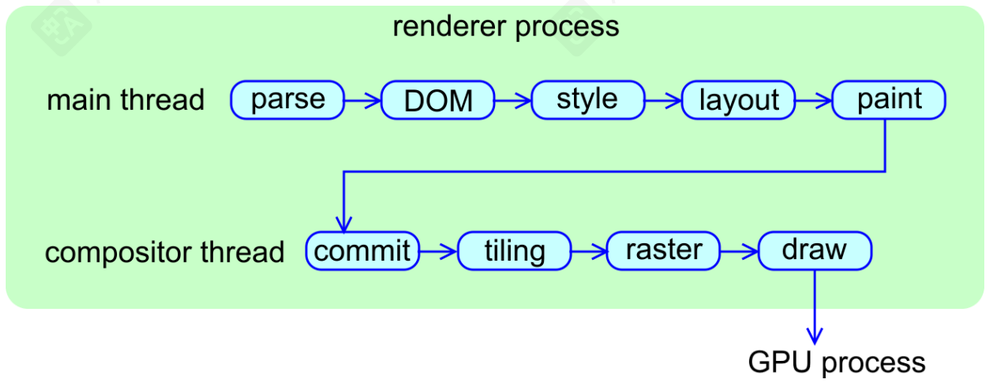
保持 V8 包装器存活有两种方式：ActiveScriptWrappable 和包装器追踪。

如果您想了解更多：

- 如何管理 V8 包装器的生命周期：bindings/core/v8/V8Wrapper.md
- 如何使用包装器追踪：platform/bindings/TraceWrapperReference.md

渲染管线

从 HTML 文件交付给 Blink 到像素在屏幕上显示，这是一段漫长的旅程。渲染管线架构如下。



阅读这份优秀的幻灯片，了解渲染管线中每个阶段的作用。（我认为我无法写出比幻灯片更好的解释了 :-)）

如果您想了解更多：

- 概述：像素的一生
- DOM: core/dom/README.md
- 样式：core/css/README.md
- 布局：core/layout/README.md
- 油漆：core/paint/README.md
- 合成器线程：Chromium 图形

Questions?

You can ask any questions to blink-dev@chromium.org (for general questions) or platform-architecture-dev@chromium.org (for architecture-related questions). We're always happy to help! :D

有问题吗?

您可以向 blink-dev@chromium.org（一般问题）或 platform-architecture-dev@chromium.org（架构相关问题）提问。我们随时乐意提供帮助！：D